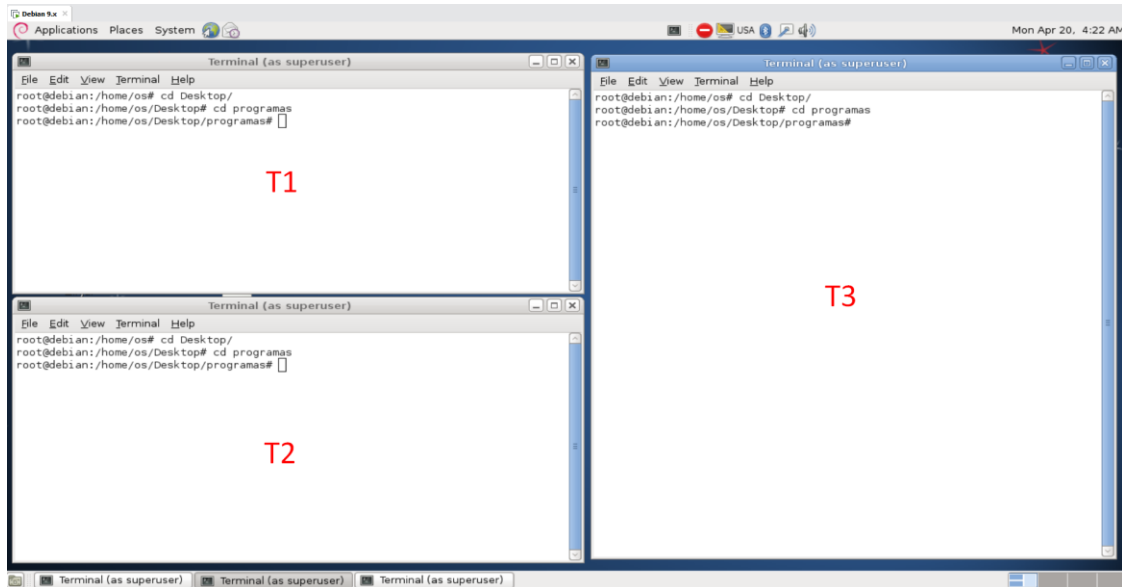


Instrucciones para hacer diferentes pruebas con los programas mostrados en la teoría

Es conveniente disponer 3 de terminales para poder apreciar bien el funcionamiento y poder monitorear su forma de operar. Además las 3 consolas deben estar en el mismo directorio donde están los programas.

Para identificar las terminales en forma abreviada, las llamaré **T1**, **T2** y **T3**:



Actividades

1) Programas **.enviaSHM.bin** y **.recibeSHM.bin**

- Ejecuta en T3, el comando `ipcs` (`ipcs` muestra información sobre las facilidades de comunicación entre procesos para las cuales el proceso de llamada tiene acceso de lectura. De manera predeterminada, muestra información sobre los tres recursos: segmentos de memoria compartida, colas de mensajes y matrices de semáforos). Verás los segmentos de memoria compartida.
- Ejecuta el programa en T2 el programa `.recibeSHM.bin`. Deberías recibir un mensaje de error de segmento, ya que no está todavía creado el segmentos de memoria compartida (esto lo hace el programa que va a enviar la información)
- Ahora prueba en ejecutar en T1, el programa `.enviaSHM.bin` y antes de ingresar la cadena a enviar, verifica en T3 con el comando `ipcs`, que se ha creado un segmento de memoria compartida (`key=0x00001234`), donde se escribirá la información a enviar.

Ahora que está creado el segmento, ejecuta en T2 el programa `.recibeSHM.bin` donde se verá que aparecen 0 indicando que se está esperando datos en la memoria compartida. Vuelve a T1 e ingresa la cadena que deseas enviar, la cual será enviada a los 10 segundos.

- Cuando se envíe la cadena, el programa que estaba esperándola muestra los datos y termina.

Notas:

- Hay que tener en cuenta que el programa que envía los datos es el encargado de eliminar el segmento de memoria (con `hmctl(shmid,IPC_RMID,0);`), por lo que si el que recibe no está esperando puede dar el error comentado en b)
- Una posible alternativa es hacer que el programa que recibe los datos sea el encargado de eliminar la cadena.
- Si por algún motivo se desea eliminar el segmento compartido desde la consola, debe utilizar el comando `ipcrm -m shmid` (`shmid` corresponde a la segunda columna del listado)

2) Programa **variablescompartidas.cpp**:

- Este programa conviene ejecutarlo desde el Zinjai
- Genera 2 hilos que invocan a una función
- Simplemente muestra el valor que toman las variables de considerando su ámbito de validez (la tabla es la que se encuentra en el Power Point)

3) Programa **pipeteoria1.cpp**:

- Este programa conviene ejecutarlo desde el Zinjai.
- Crea una tubería sin nombre y luego solicita el ingreso de un texto.
- Crea un proceso hijo donde el padre luego de esperar 2 segundos le envía el texto al hijo y espera a que muestre el texto y termine. Apenas se crea el proceso hijo espera 3 segundos. Como está ejecutándose en forma paralela con el padre ambos terminarán luego de 3 segundos.

4) Programas **./productor.bin** y **./consumidor.bin**

- Ejecuta en T1 el siguiente programa `./productor.bin SistemasOperativos` (después de nombre del programa va una cadena sin espacios). Al pulsar enter, el programa espera que el consumidor se ejecute.
- El productor crea una tubería con nombre (en este caso se llama "fifo") y la puedes ver si en la consola T3 escribes `ls -l` donde estará este archivo:

```
prw-r--r-- 1 root root      0 Apr 20 04:59 fifo
```
- Ahora en T2 escribe `./consumidor.bin` y automáticamente recibirá la información enviada por el productor y ambos programas finalizarán. Este programa elimina la tubería.

Notas:

- Si se ejecuta el consumidor antes de que el productor cree la tubería dará un error de apertura de tubería.
- Si la tubería estaba creada y el productor intenta crearla dará una error de creación de tubería (verificar con el punto b) y si esta está, se la puede borrar con `unlink fifo` o como con cualquier archivo `rm fifo`)
- Si una tubería estaba creada, y se ejecuta el consumidor, este esperará a que el productor envíe datos.

5) Programas **./enviamsg.bin** y **./recibemsg.bin**

- Estos programas utilizan colas de mensajes
- Ejecuta en T3, el comando `ipcs` para ver si hay creada una cola de mensajes

```
----- Message Queues -----
key          msqid      owner          perms        used-bytes   messages
```

- Ejecuta el programa en T2 el programa `./recibemsg.bin`. Deberías recibir un mensaje de error -1, ya que no está todavía no hay ninguna cola de mensajes (esto lo hace el programa que va a enviar la información)
- Ahora prueba en ejecutar en T1, el programa `./enviamsg.bin` y debería avisar que el mensaje se envió con éxito. Puedes enviar todos los mensajes que quieras desde el Zinjai cambiándolos y ejecutando el programa (no hace falta que lo hagas desde la línea de comandos)
- Verifica nuevamente el punto b).

```
----- Message Queues -----
key          msqid      owner          perms        used-bytes   messages
0x00001234  32768      root           666           25           1
```

- f. Ahora ejecuta en T2 el programa `./recibemsg.bin` y obtendrás el mensaje de la cola siguiendo el orden de ingreso (fifo)

Nota:

- La variable `mtype` del struct puede permitir variar el tipo de mensaje para que en la recepción de los datos se realice de acuerdo a algún orden en particular (ver documentación subida)-
- Si se desea eliminar una cola de mensajes es con: `ipcrm -q msqid`

6) Programa `./concurrency.bin`

- a. Este programa pone de manifiesto el problema que pueden tener varios hilos que se ejecutan en forma concurrente y acceden a una misma variable. Si las funciones de los hilos se ejecutaran una después de otra o en ningún momento se accediera al recurso compartido (sección crítica) en ambas funciones en forma simultánea el programa debería dar como resultado el doble de parámetro. Si el resultado cambia con cada ejecución, se debe a la condición de carrera y lo que está sucediendo es lo que se ve en la página 17 del PowerPoint de comunicación, concurrencia y sincronización.
- b. Prueba primero el programa con un número pequeño por ejemplo:
`./concurrency.bin 100000` deberías obtener en sum: 200000
Dependiendo la computadora puede variar el valor (algunas pueden necesitar valores más altos) pero por ejemplo en mi PC virtual, si probamos con `./concurrency.bin 4000000` obtengo sum: 6314265 en lugar de 8000000 lo que está indicando que hubo 8000000-6314265 situaciones en donde se accedió en ambas secciones críticas a la vez y se dejó de sumar un valor (condición de carrera como lo menciona en el punto a) y con cada ejecución puede dar diferentes valores (por casualidad podría dar el resultado correcto).

```
root@debian:/home/os/Desktop/programas# ./concurrency.bin 100000
Valor inicial de sum: 0   direccion de sum: 0x8049f74
comienza funcion: B   direccion de i: 0xb6cd2378   direccion de sum: 0x8049f74
fin funcion B
comienza funcion: A   direccion de i: 0xb74d3378   direccion de sum: 0x8049f74
fin funcion A
sum: 200000
deberia dar: 200000

root@debian:/home/os/Desktop/programas# ./concurrency.bin 4000000
Valor inicial de sum: 0   direccion de sum: 0x8049f74
comienza funcion: B   direccion de i: 0xb6ca5378   direccion de sum: 0x8049f74
comienza funcion: A   direccion de i: 0xb74a6378   direccion de sum: 0x8049f74
fin funcion B
fin funcion A
sum: 6314265
deberia dar: 8000000
```

7) Programa `./livelock.bin`

- a. Este programa muestra el problema de bloqueo entre procesos livelock (el cual no es un deadlock).
- b. Se ejecutará hasta que se den las condiciones (si es que se dan para que solo salga) por ejemplo al ejecutar `./livelock.bin` en una ejecución dio vueltas 5353 veces hasta que pudo salir del ciclo de entrada entre los dos hilos y ejecutar la sección crítica, mientras que en otra corrida estuvo 30471 veces:

```
3250) espera f2.....
5251) espera f1.....
5252) espera f2.....
5253) espera f1.....
seccion critica de f2
seccion critica de f1
suma: 20
root@debian:/home/os/Desktop/programas#

30468) espera f2.....
30469) espera f2.....
30470) espera f1.....
30471) espera f2.....
seccion critica de f1
seccion critica de f2
suma: 20
root@debian:/home/os/Desktop/programas#
```

8) Programa `sincropeterson0.cpp`

- a. Ejecutar el programa desde el Zinjai. En el mismo se puede visualizar el algoritmo de sincronización de Peterson, en el cual se puede ver la espera activa de cada función

(ciclos while de las líneas 20 y 38) y cada función va a esperar mientras el el turno sea el de la función pero la bandera sea de la otra.

- b. Mientras que una función está en su sección crítica, la otra está esperando.
- c. Finalizar con Ctrl+C

Muchos ejemplos en internet: <https://youtu.be/RRaP8pTU0zI>

9) Programa **ABC.cpp**

- a. Ejecutar el programa desde el Zinjai. Este programa muestra como se puede sincronizar la secuencia de ejecución de los hilos con el uso de semáforos para que escriban siempre la misma secuencia y así evitar las condiciones de carrera.
- b. Se crean 3 semáforos de tal manera que cuando termina una función activa el semáforo de la siguiente y así sucesivamente. Es importante que uno de los semáforos tenga inicializada la señal de comienzo para que no se produzca un deadlock. La flecha verde indica el semáforo que iniciará el hilo (sem_A) (la analogía sería cual empieza estando en verde)

```
54 sem_init(&sem_A, 0, 1);  
55 sem_init(&sem_B, 0, 0);  
56 sem_init(&sem_C, 0, 0);  
--
```

- c. Finalizar con Ctrl+C